

Clinic Appointment & Patient Records API

Test Report

Name: Ya'Rab Mohammed Al-Mamari

CLINIC MANAGEMENT SYSTEM - POSTMAN TEST CASES

Base URL: <http://localhost:8080>

1. CREATE DOCTOR

URL: **POST /api/doctors**

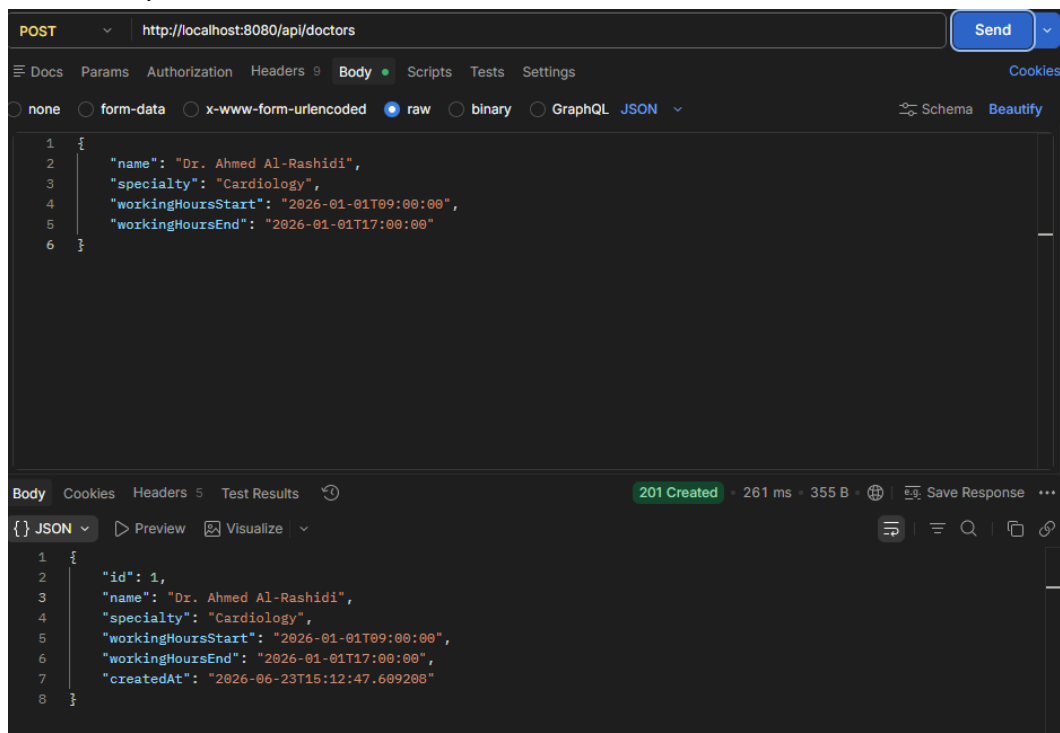
Request Body (JSON):

```
{
  "name": "Dr. Ahmed Al-Rashidi",
  "specialty": "Cardiology",
  "workingHoursStart": "2026-01-01T09:00:00",
  "workingHoursEnd": "2026-01-01T17:00:00"
}
```

Expected Response: **201 Created**

```
{
  "id": 1,
  "name": "Dr. Ahmed Al-Rashidi",
  "specialty": "Cardiology",
  "workingHoursStart": "2026-01-01T09:00:00",
  "workingHoursEnd": "2026-01-01T17:00:00",
  "createdAt": "2026-06-22T... (timestamp)"
}
```

Actual Response: **201 Created**



2. CREATE PATIENT

URL: **POST /api/patients**

Request Body (JSON):

```
{
  "name": "Sara Al-Mutairi",
  "dateOfBirth": "1990-05-15",
  "phone": "966501234567"
}
```

Expected Response: **201 Created**

```
{
  "id": 1,
  "name": "Sara Al-Mutairi",
  "dateOfBirth": "1990-05-15",
  "phone": "966501234567",
  "createdAt": "2026-06-22T... (timestamp)"
}
```

Actual Response: **201 Created**

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/patients
- Request Body (JSON):**

```
{
  "name": "Sara Al-Mutairi",
  "dateOfBirth": "1990-05-15",
  "phone": "966501234567"
}
```
- Response:** 201 Created (28 ms, 293 B)
- Response Body (JSON):**

```
{
  "id": 1,
  "name": "Sara Al-Mutairi",
  "dateOfBirth": "1990-05-15",
  "phone": "966501234567",
  "createdAt": "2026-06-23T15:14:29.962401"
}
```

3. GENERATE SLOTS

URL: **POST /api/doctors/1/slots**

Request Body (JSON):

```
{
  "date": "2026-07-01"
}
```

Expected Response: 201 Created

```
[
  {
    "id": 1,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-15T09:00:00",
    "endTime": "2026-07-15T09:30:00",
    "status": "FREE"
  },
  {
    "id": 2,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-15T09:30:00",
    "endTime": "2026-07-15T10:00:00",
    "status": "FREE"
  },
  ... (all 30-min slots from 09:00 to 17:00 = 16 slots total)
]
```

Actual Response: **201 Created**

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/doctors/1/slots
- Body:** raw, containing the JSON:

```
{
  "date": "2026-07-01"
}
```
- Status:** 201 Created (305 ms, 1.97 KB)
- Response Body:** JSON array containing 5 slot objects:

```
[
  {
    "id": 1,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:00:00",
    "endTime": "2026-07-01T09:30:00",
    "status": "FREE"
  },
  {
    "id": 2,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:30:00",
    "endTime": "2026-07-01T10:00:00",
    "status": "FREE"
  },
  {
    "id": 3,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T10:00:00",
    "endTime": "2026-07-01T10:30:00",
    "status": "FREE"
  },
  {
    "id": 4,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T10:30:00",
    "endTime": "2026-07-01T11:00:00",
    "status": "FREE"
  },
  {
    "id": 5,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T11:00:00",
    "endTime": "2026-07-01T11:30:00",
    "status": "FREE"
  }
]
```

4. VIEW SCHEDULE

URL: **GET /api/doctors/1/schedule?date=2026-07-01**

Expected Response: **200 OK**

```
[
  {
    "id": 1,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-15T09:00:00",
    "endTime": "2026-07-15T09:30:00",
    "status": "FREE"
  },
  ...
  (same as slot generation output)
]
```

Actual Response: **200 OK**

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/doctors/1/schedule?date=2026-07-01
- Response Status:** 200 OK
- Response Time:** 60 ms
- Response Size:** 1.96 KB
- Response Body (JSON):**

```
[
  {
    "id": 1,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:00:00",
    "endTime": "2026-07-01T09:30:00",
    "status": "FREE"
  },
  {
    "id": 2,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:30:00",
    "endTime": "2026-07-01T10:00:00",
    "status": "FREE"
  },
  {
    "id": 3,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T10:00:00",
    "endTime": "2026-07-01T10:30:00",
    "status": "FREE"
  }
]
```

5. BOOK APPOINTMENT

URL: **POST /api/appointments**

Request Body (JSON):

```
{
  "slotId": 1,
  "patientId": 1
}
```

Expected Response: **201 Created**

```
{
  "id": 1,
  "slotId": 1,
  "patientId": 1,
  "patientName": "Sara Al-Mutairi",
  "doctorName": "Dr. Ahmed Al-Rashidi",
  "slotDate": "2026-07-15",
  "startTime": "2026-07-15T09:00:00",
  "endTime": "2026-07-15T09:30:00",
  "status": "BOOKED",
  "bookedAt": "2026-06-22T... (timestamp)",
  "cancelledAt": null,
  "completedAt": null,
  "rescheduledToId": null
}
```

Actual Response: **201 Created**

The screenshot displays a REST client interface with the following details:

- Request:** Method **POST**, URL **http://localhost:8080/api/appointments**. The request body is a JSON object:

```
{
  "slotId": 1,
  "patientId": 1
}
```
- Response:** Status **201 Created**, time **84 ms**, size **480 B**. The response body is a JSON object:

```
{
  "id": 1,
  "slotId": 1,
  "patientId": 1,
  "patientName": "Sara Al-Mutairi",
  "doctorName": "Dr. Ahmed Al-Rashidi",
  "slotDate": "2026-07-01",
  "startTime": "2026-07-01T09:00:00",
  "endTime": "2026-07-01T09:30:00",
  "status": "BOOKED",
  "bookedAt": "2026-06-23T15:19:05.2929499",
  "cancelledAt": null,
  "completedAt": null,
  "rescheduledToId": null
}
```

6. CANCEL APPOINTMENT

URL: **POST** /api/appointments/1/cancel

Request Body: (none / empty)

Expected Response: **200 OK**

```
{
  "id": 1,
  "slotId": 1,
  "patientId": 1,
  "patientName": "Sara Al-Mutairi",
  "doctorName": "Dr. Ahmed Al-Rashidi",
  "slotDate": "2026-07-15",
  "startTime": "2026-07-15T09:00:00",
  "endTime": "2026-07-15T09:30:00",
  "status": "CANCELLED",
  "bookedAt": "2026-06-22T...",
  "cancelledAt": "2026-06-22T... (timestamp)",
  "completedAt": null,
  "rescheduledToId": null
}
```

Actual Response: **200 OK**

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/appointments/1/cancel
- Body:** none
- Response:** 200 OK, 39 ms, 501 B
- Response Body (JSON):**

```
1 {
2   "id": 1,
3   "slotId": 1,
4   "patientId": 1,
5   "patientName": "Sara Al-Mutairi",
6   "doctorName": "Dr. Ahmed Al-Rashidi",
7   "slotDate": "2026-07-01",
8   "startTime": "2026-07-01T09:00:00",
9   "endTime": "2026-07-01T09:30:00",
10  "status": "CANCELLED",
11  "bookedAt": "2026-06-23T15:19:05.29295",
12  "cancelledAt": "2026-06-23T15:20:47.7603853",
13  "completedAt": null,
14  "rescheduledToId": null
15 }
```

7. RESCHEDULE APPOINTMENT

Reschedule appointment with id 2 to a new slot:

URL: **POST /api/appointments/2/reschedule**

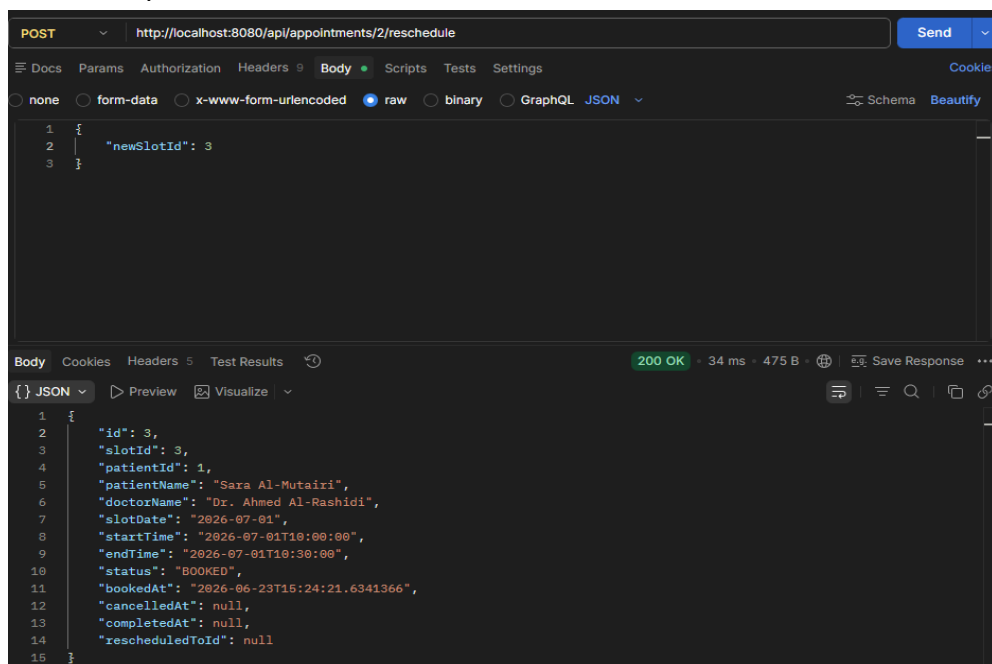
Request Body (JSON):

```
{
  "newSlotId": 3
}
```

Expected Response: **200 OK**

```
{
  "id": 3,
  "slotId": 3,
  "patientId": 1,
  "patientName": "Sara Al-Mutairi",
  "doctorName": "Dr. Ahmed Al-Rashidi",
  "slotDate": "2026-07-15",
  "startTime": "2026-07-15T10:00:00",
  "endTime": "2026-07-15T10:30:00",
  "status": "BOOKED",
  "bookedAt": "2026-06-22T...",
  "cancelledAt": null,
  "completedAt": null,
  "rescheduledToId": null
}
```

Actual Response: **201 Created**



NOTE: The OLD appointment (id=2) now has status "RESCHEDULED" and `rescheduledToId` = 3 (the new appointment id).

=====

8. RECORD VISIT

=====

PREREQUISITE: Need a BOOKED appointment whose slot end_time has already passed.

Record visit for appointment with id 4 to a new slot:

URL: **POST /api/appointments/4/visit**

Request Body (JSON):

```
{
  "diagnosis": "Patient shows signs of high blood pressure. Prescribed
medication.",
  "prescription": "Lisinopril 10mg daily for 30 days"
}
```

Expected Response: **201 Created**

```
{
  "id": 1,
  "appointmentId": 4,
  "diagnosis": "Patient shows signs of high blood pressure. Prescribed
medication.",
  "prescription": "Lisinopril 10mg daily for 30 days",
  "recordedAt": "2026-06-22T... (timestamp)"
}
```

Actual Response: **201 Created**

The appointment (id=4) now has status "COMPLETED" and completedAt set.

9. PATIENT HISTORY

URL: **GET /api/patients/1/history**

Expected Response: **200 OK**

```
[
  {
    "appointmentId": 4,
    "slotDate": "2026-06-01",
    "startTime": "2026-06-01T09:00:00",
    "endTime": "2026-06-01T09:30:00",
    "doctorName": "Dr. Ahmed Al-Rashidi",
    "doctorSpecialty": "Cardiology",
    "status": "COMPLETED",
    "diagnosis": "Patient shows signs of high blood pressure...",
    "prescription": "Lisinopril 10mg daily for 30 days",
    "bookedAt": "2026-06-22T...",
    "cancelledAt": null,
    "completedAt": "2026-06-22T..."
  },
  {
    "appointmentId": 3,
    "slotDate": "2026-07-15",
    "startTime": "2026-07-15T10:00:00",
    "endTime": "2026-07-15T10:30:00",
    "doctorName": "Dr. Ahmed Al-Rashidi",
    "doctorSpecialty": "Cardiology",
    "status": "BOOKED",
    "diagnosis": null,
    "prescription": null,
    ...
  },
  ... (all appointments for patient, ordered by date descending)
]
```

Actual Response: **200 OK**

The screenshot shows a web browser interface with the URL `http://localhost:8080/api/patients/1/history` and the method `GET`. The response is displayed in JSON format, showing a list of appointments for patient 1, ordered by date descending. The response status is **200 OK**.

```
[
  {
    "appointmentId": 1,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:00:00",
    "endTime": "2026-07-01T09:30:00",
    "doctorName": "Dr. Ahmed Al-Rashidi",
    "doctorSpecialty": "Cardiology",
    "status": "CANCELLED",
    "diagnosis": null,
    "prescription": null,
    "bookedAt": "2026-06-23T15:19:05.29295",
    "cancelledAt": "2026-06-23T15:20:47.760385",
    "completedAt": null
  },
  {
    "appointmentId": 3,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T10:00:00",
    "endTime": "2026-07-01T10:30:00",
    "doctorName": "Dr. Ahmed Al-Rashidi",
    "doctorSpecialty": "Cardiology",
    "status": "RESCHEDULED",
    "diagnosis": null,
    "prescription": null,
    "bookedAt": "2026-06-23T15:24:21.634137",
    "cancelledAt": null,
    "completedAt": null
  },
  {
    "appointmentId": 4,
    "slotDate": "2026-07-01",
    "startTime": "2026-07-01T09:30:00",
    "endTime": "2026-07-01T10:00:00",
    "doctorName": "Dr. Ahmed Al-Rashidi",
    "doctorSpecialty": "Cardiology",
    "status": "COMPLETED",
    "diagnosis": "Patient shows signs of high blood pressure...",
    "prescription": "Lisinopril 10mg daily for 30 days",
    "bookedAt": "2026-06-22T...",
    "cancelledAt": null,
    "completedAt": "2026-06-22T..."
  }
]
```

BUSINESS RULE VIOLATION TESTS

10. DOUBLE-BOOKING A SLOT

PREREQUISITE: Have a BOOKED appointment for slot 2.

URL: **POST** /api/appointments

Request Body (JSON):

```
{
  "slotId": 2,
  "patientId": 1
}
```

Expected Response: 409 Conflict

```
{
  "message": "Slot is already booked",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: 409 Conflict

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments`. The request body is a JSON object: `{ "slotId": 2, "patientId": 1 }`. The response is a `409 Conflict` status with a message: `"Slot is already booked"`, a status of `409`, and a timestamp: `"2026-06-23T15:42:36.382397300"`. The response is displayed in JSON format with syntax highlighting.

```
POST http://localhost:8080/api/appointments
{
  "slotId": 2,
  "patientId": 1
}
```

409 Conflict • 15 ms • 263 B • Save Response

```
{
  "message": "Slot is already booked",
  "status": 409,
  "timestamp": "2026-06-23T15:42:36.382397300"
}
```

11. TWO ACTIVE APPOINTMENTS SAME DOCTOR SAME DAY

PREREQUISITE: Have patient 1 with a BOOKED appointment for slot 2 (doctor 1).

Book another slot (id=2, same doctor, same date):

URL: **POST /api/appointments**

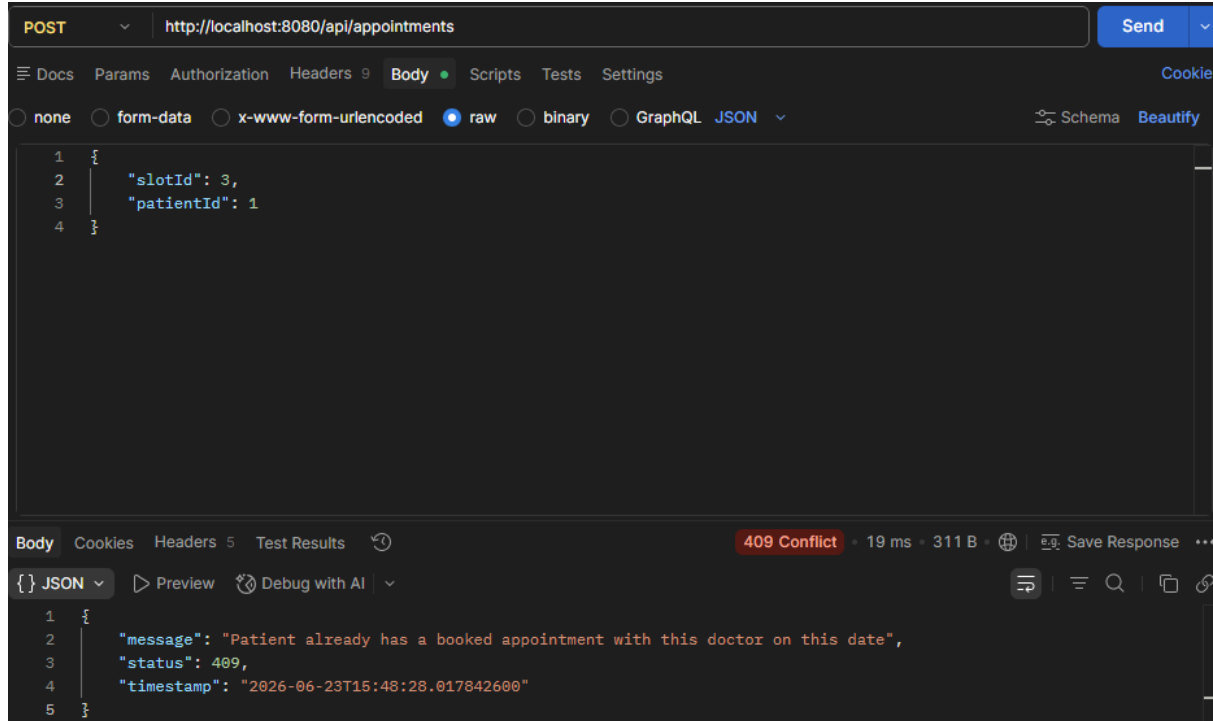
Request Body (JSON):

```
{
  "slotId": 3,
  "patientId": 1
}
```

Expected Response: 409 Conflict

```
{
  "message": "Patient already has a booked appointment with this doctor on this date",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **409 Conflict**



NOTE: This will only trigger if you book ANOTHER slot for the SAME doctor on the SAME date. Slot IDs 1 and 2 are both for doctor 1 on 2026-07-15.

12. RECORD VISIT FOR FUTURE APPOINTMENT

PREREQUISITE: Have a BOOKED appointment (id=3) for a FUTURE slot (slot end_time hasn't passed yet, e.g. slot id=3 on 2026-07-15).

URL: **POST** /api/appointments/4/visit

Request Body (JSON):

```
{
  "diagnosis": "Test diagnosis",
  "prescription": "Test prescription"
}
```

Expected Response: **409 Conflict**

```
{
  "message": "Cannot record visit: appointment slot time has not yet passed",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **409 Conflict**

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/appointments/4/visit
- Body (Request):** A JSON object with "diagnosis": "Test diagnosis".
- Status:** 409 Conflict
- Response Body (JSON):**

```
{
  "message": "Cannot record visit: appointment slot time has not yet passed",
  "status": 409,
  "timestamp": "2026-06-23T15:52:18.713400100"
}
```

13. RECORD VISIT FOR CANCELLED APPOINTMENT

PREREQUISITE: Have a CANCELLED appointment (e.g. id=1 after cancelling).

URL: **POST /api/appointments/1/visit**

Request Body (JSON):

```
{
  "diagnosis": "Test diagnosis"
}
```

Expected Response: **409 Conflict**

```
{
  "message": "Visit can only be recorded for booked appointments",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **409 Conflict**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments/1/visit`. The request body is a JSON object: `{ "diagnosis": "Test diagnosis" }`. The response is a **409 Conflict** status with a message: `"Visit can only be recorded for booked appointments"`, status `409`, and a timestamp: `"2026-06-23T15:53:40.633287300"`.

POST `http://localhost:8080/api/appointments/1/visit` Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookie

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "diagnosis": "Test diagnosis"
3 }
```

Body Cookies Headers 5 Test Results 409 Conflict 8 ms 291 B Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Visit can only be recorded for booked appointments",
3   "status": 409,
4   "timestamp": "2026-06-23T15:53:40.633287300"
5 }
```

ERROR HANDLING TESTS

14. VALIDATION ERROR - MISSING DOCTOR NAME

URL: **POST /api/doctors**

Request Body (JSON):

```
{
  "specialty": "Cardiology",
  "workingHoursStart": "2026-01-01T09:00:00",
  "workingHoursEnd": "2026-01-01T17:00:00"
}
```

Expected Response: **400 Bad Request**

```
{
  "message": "Validation failed: name: Doctor name is required",
  "status": 400,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **400 Bad Request**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/doctors`. The request body is a JSON object with the following fields:

```
{
  "specialty": "Cardiology",
  "workingHoursStart": "2026-01-01T09:00:00",
  "workingHoursEnd": "2026-01-01T17:00:00"
}
```

The response is a **400 Bad Request** with a status of 400. The response body is a JSON object with the following fields:

```
{
  "message": "Validation failed: name Doctor name is required",
  "status": 400,
  "timestamp": "2026-06-23T16:06:53.070369100"
}
```

15. VALIDATION ERROR - WORKING HOURS START AFTER END

URL: **POST /api/doctors**

Request Body (JSON):

```
{
  "name": "Test Doctor",
  "specialty": "General",
  "workingHoursStart": "2026-01-01T17:00:00",
  "workingHoursEnd": "2026-01-01T09:00:00"
}
```

Expected Response: **400 Bad Request**

```
{
  "message": "Start time must be before end time",
  "status": 400,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **400 Bad Request**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/doctors`. The request body is a JSON object with the following fields: `"name": "Test Doctor", "specialty": "General", "workingHoursStart": "2026-01-01T17:00:00", "workingHoursEnd": "2026-01-01T09:00:00"`. The response is a **400 Bad Request** with a status of 400 and a message: `"message": "Start time must be before end time", "status": 400, "timestamp": "2026-06-23T16:07:47.673617300"`. The response is displayed in the JSON tab, and the status bar indicates a 400 Bad Request with a 7 ms response time and 249 B of data.

```
POST http://localhost:8080/api/doctors
{
  "name": "Test Doctor",
  "specialty": "General",
  "workingHoursStart": "2026-01-01T17:00:00",
  "workingHoursEnd": "2026-01-01T09:00:00"
}
```

```
{
  "message": "Start time must be before end time",
  "status": 400,
  "timestamp": "2026-06-23T16:07:47.673617300"
}
```

16. NOT FOUND - BOOK APPOINTMENT WITH INVALID SLOT

URL: **POST** /api/appointments

Request Body (JSON):

```
{
  "slotId": 999,
  "patientId": 1
}
```

Expected Response: **404 Not Found**

```
{
  "message": "Slot not found or not available",
  "status": 404,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **404 Not Found**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments`. The request body is a JSON object: `{ "slotId": 999, "patientId": 1 }`. The response is a **404 Not Found** status with a response time of 13 ms and a body size of 273 B. The response body is a JSON object: `{ "message": "Slot not found or not available", "status": 404, "timestamp": "2026-06-23T16:09:12.228305700" }`.

POST `http://localhost:8080/api/appointments` **Send**

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "slotId": 999,
3   "patientId": 1
4 }
```

Body Cookies Headers 5 Test Results **404 Not Found** • 13 ms • 273 B • Save Response ...

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Slot not found or not available",
3   "status": 404,
4   "timestamp": "2026-06-23T16:09:12.228305700"
5 }
```

17. NOT FOUND - INVALID PATIENT

URL: **POST /api/appointments**

Request Body (JSON):

```
{
  "slotId": 1,
  "patientId": 999
}
```

Expected Response: **404 Not Found**

Actual Response: **404 Not Found**

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/appointments
- Body:** JSON
- Request Body (JSON):**

```
{
  "slotId": 1,
  "patientId": 999
}
```
- Response:** 404 Not Found (10 ms, 272 B)
- Response Body (JSON):**

```
{
  "message": "Patient not found with id: 999",
  "status": 404,
  "timestamp": "2026-06-23T16:09:53.897180800"
}
```

18. NOT FOUND - INVALID DOCTOR SCHEDULE

URL: **GET /api/doctors/999/schedule?date=2026-07-15**

Expected Response: **404 Not Found**

Actual Response: **404 Not Found**

The screenshot displays a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8080/api/doctors/999/schedule?date=2026-07-15
- Request Body (raw):**

```
1 {  
2   "slotId": 1,  
3   "patientId": 999  
4 }
```
- Response Status:** 404 Not Found (12 ms, 271 B)
- Response Body (JSON):**

```
1 {  
2   "message": "Doctor not found with id: 999",  
3   "status": 404,  
4   "timestamp": "2026-06-23T16:11:10.944542100"  
5 }
```

19. VALIDATION - EMPTY DIAGNOSIS

URL: **POST /api/appointments/4/visit**

Request Body (JSON):

```
{
  "prescription": "Some prescription"
}
```

Expected Response: **400 Bad Request**

Actual Response: **400 Bad Request**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments/4/visit`. The request body is a JSON object: `{ "prescription": "Some prescription" }`. The response is a `400 Bad Request` with a status bar indicating `9 ms` and `265 B`. The response body is a JSON object: `{ "message": "Validation failed: diagnosis Diagnosis is required", "status": 400, "timestamp": "2026-06-23T16:12:45.368852200" }`.

```
POST http://localhost:8080/api/appointments/4/visit
{
  "prescription": "Some prescription"
}
```

400 Bad Request • 9 ms • 265 B • Save Response

```
{
  "message": "Validation failed: diagnosis Diagnosis is required",
  "status": 400,
  "timestamp": "2026-06-23T16:12:45.368852200"
}
```

20. CANCEL ALREADY CANCELLED APPOINTMENT

URL: **POST /api/appointments/1/cancel**

Request Body: (none)

Expected Response: **409 Conflict**

```
{
  "message": "Only BOOKED appointments can be cancelled",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **409 Conflict**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments/1/cancel`. The request body is set to 'raw' and contains a JSON object: `{ "prescription": "Some prescription" }`. The response is a **409 Conflict** status with a response time of 10 ms and a size of 282 B. The response body is displayed in JSON format: `{ "message": "Only BOOKED appointments can be cancelled", "status": 409, "timestamp": "2026-06-23T16:13:47.983841288" }`.

POST `http://localhost:8080/api/appointments/1/cancel` **Send**

Docs Params Authorization Headers 9 **Body** Scripts Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON ☐ Schema Beautify

```
1 {
2   |   "prescription": "Some prescription"
3   | }
```

Body Cookies Headers 5 Test Results **409 Conflict** • 10 ms • 282 B • Save Response

{ } JSON Preview Debug with AI

```
1 {
2   |   "message": "Only BOOKED appointments can be cancelled",
3   |   "status": 409,
4   |   "timestamp": "2026-06-23T16:13:47.983841288"
5   | }
```

21. SLOT DATE IN THE PAST

URL: **POST /api/doctors/1/slots**

Request Body (JSON):

```
{
  "date": "2020-01-01"
}
```

Expected Response: **400 Bad Request**

```
{
  "message": "Slot date must be in the future",
  "status": 400,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **400 Bad Request**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/doctors/1/slots`. The request body is a JSON object: `{ "date": "2020-01-01" }`. The response is a `400 Bad Request` with a status of 400 and a message: `"Slot date must be in the future"`. The response timestamp is `"2026-06-23T16:15:19.558594900"`.

POST `http://localhost:8080/api/doctors/1/slots` Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookie

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ☐ Schema Beautify

```
1 {
2   "date": "2020-01-01"
3 }
```

Body Cookies Headers 4 Test Results 400 Bad Request · 11 ms · 246 B · Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "message": "Slot date must be in the future",
3   "status": 400,
4   "timestamp": "2026-06-23T16:15:19.558594900"
5 }
```

22. RESCHEDULE NON-BOOKED APPOINTMENT

URL: **POST /api/appointments/1/reschedule**

Request Body (JSON):

```
{
  "newSlotId": 5
}
```

Expected Response: **409 Conflict**

```
{
  "message": "Only BOOKED appointments can be rescheduled",
  "status": 409,
  "timestamp": "2026-06-22T..."
}
```

Actual Response: **409 Conflict**

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/appointments/1/reschedule`. The request body is a JSON object: `{ "newSlotId": 5 }`. The response is a **409 Conflict** status with a message: `"Only BOOKED appointments can be rescheduled"`, status `409`, and a timestamp: `"2026-06-23T16:16:24.625190100"`.

Request Details:

- Method: POST
- URL: `http://localhost:8080/api/appointments/1/reschedule`
- Body:

```
{
  "newSlotId": 5
}
```

Response Details:

- Status: 409 Conflict
- Message: `"Only BOOKED appointments can be rescheduled"`
- Status Code: `409`
- Timestamp: `"2026-06-23T16:16:24.625190100"`